

四電一丙 數位邏輯實習

第九次實習作業報告

組員1 學號:114310329 姓名:許宥翔, 組員2 學號: B812111001姓名:張博翔

組員1 於此作業中的貢獻:解題(作業二)、*digital*、*Quartus*、拍照(作業二)、作業報告(第二部分)

組員2 於此作業中的貢獻:解題(作業一)、*digital*、*quartus*、拍照(作業一)、作業報告(第一部分)

I. 實驗目的

A. 作業1

作業題1:利用Mealy狀態機來設計一個電路, 電路的輸入輸出分別為一位元的X與Z。此電路會於每四次觸發後, 檢查四位元區塊是否為一個有效的十進位數字之BCD表示;若是有效的BCD, 則令Z=1。本電路需要每四次觸發之後回到S0進行重製, 狀態表必須利用本次上課的方式(含梯形圖)進行最佳化。

電路請從以下幾種方法挑選兩種方式來完成:

- VHDL Process設計模式
- VHDL Component設計模式
- VHDL 布林代數式設計模式
- Digital 拉電路

B. 作業2

作業題2:實做四位元Serial Adder電路, 控制電路請以Moore狀態機完成。投影片22與26頁的控制電路採用Mealy狀態機, 本題請重新設計成Moore狀態機。電路不限定設計方式

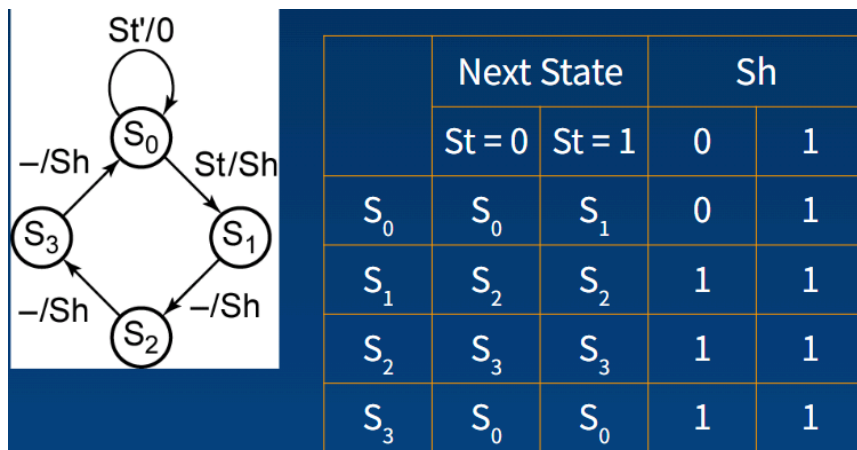


圖1:mealy機

II. 設計程序

A. 作業1

1. 利用Mealy狀態機來設計單次輸入1 bit 的 BCD detector (sequential), 以State表示序列輸入的狀態, 由於Mealy的輸出由當下State與輸入X決定, State最多表示到111, 表示輸入3個1, 如下圖所示:

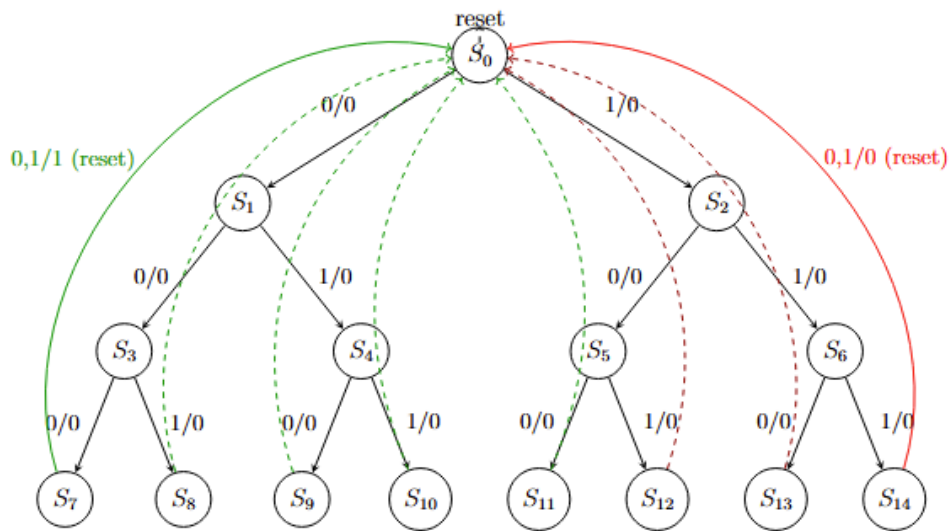


圖2:BCD detector state graph (未簡化)

圖中S0為初始狀態，輸入4次後reset，其中綠色線代表符合BCD；反之紅色代表無效範圍。

有效條件: $b_3 = 0$ (所有 0xxx 皆有效)，或 $b_3b_2b_1 = 100$ (即 S8 或 S9)，輸入順序為 MSB 先輸入，即依序輸入 $b_3 \rightarrow b_2 \rightarrow b_1 \rightarrow b_0$ 。

- 列出state table，可以發現S7~S11以及S12~S14由於下一個狀態以及輸出相同，因此可以合併狀態

表1. BCD detector original state table (含冗餘)

輸入序列	現在狀態	NS (X=0)	NS (X=1)	Z (X=0)	Z (X=1)
reset	S_0	S_1	S_2	0	0
0	S_1	S_3	S_4	0	0
1	S_2	S_5	S_6	0	0
00	S_3	S_7	S_8	0	0
01	S_4	S_9	S_{10}	0	0
10	S_5	S_{11}	S_{12}	0	0
11	S_6	S_{13}	S_{14}	0	0
000	S_7	S_0	S_0	1	1
001	S_8	S_0	S_0	1	1
010	S_9	S_0	S_0	1	1
011	S_{10}	S_0	S_0	1	1
100	S_{11}	S_0	S_0	1	1
101	S_{12}	S_0	S_0	0	0
110	S_{13}	S_0	S_0	0	0
111	S_{14}	S_0	S_0	0	0

將state table化簡，(以下將狀態轉為英文A~H表示)(可以不用轉符號，依習慣而定)

$S_7 \sim S_{11}$: 次態同為 S_0 ，輸出同為 1/1，合併 $\rightarrow G = S_7 \equiv S_8 \equiv S_9 \equiv S_{10} \equiv S_{11}$

$S_{12} \sim S_{14}$: 次態同為 S_0 ，輸出同為 0/0，可直接合併 $\rightarrow H = S_{12} \equiv S_{13} \equiv S_{14}$

又 S_3 與 S_4 在合併後，次態同為 G/G、輸出同為 0/0，故 $S_3 \equiv S_4 = D$

最後得到化簡後的 8 個狀態 ($A=S_0, B=S_1, C=S_2, D=S_3/S_4, E=S_5, F=S_6, G=S_7 \equiv S_8 \equiv S_9 \equiv S_{10} \equiv S_{11}, H=S_{12} \equiv S_{13} \equiv S_{14}$)

表2. BCD detector reduced state table

狀態		NS ($X=0$)	NS ($X=1$)	Z ($X=0$)	Z ($X=1$)
A	Reset	B	C	0	0
B	收到 $b_3=0$	D	D	0	0
C	收到 $b_3=1$	E	F	0	0
D	收到 $b_3b_2 \in \{0x\}$ ，必有效	G	G	0	0
E	收到 $b_3b_2=10$ ，部分有效	G	H	0	0
F	收到 $b_3b_2=11$ ，必無效	H	H	0	0
G	有效 BCD 完成	A	A	1	1
H	無效 BCD 完成	A	A	0	0

3. 由於該方式並非絕對能將狀態化為最簡，因此還需對化簡後的狀態建立梯形圖，判斷是否存在更多等效狀態。

	A	B	C	D	E	F	G
B	×						
	$D \equiv B, C \equiv D$						
C	×	×					
	$B \equiv E, C \equiv F$	$D \equiv E, D \equiv F$					
D	×	×	×				
	$B \equiv G, C \equiv G$	$D \equiv G$	$E \equiv G, F \equiv G$				
E	×	×	×	×			
	$B \equiv G, C \equiv H$	$D \equiv G, D \equiv H$	$E \equiv G, F \equiv H$	$G \equiv H$			
F	×	×	×	×	×		
	$B \equiv H, C \equiv H$	$D \equiv H$	$E \equiv H, F \equiv H$	$G \equiv H$	$G \equiv H$		
G	×	×	×	×	×	×	
H	×	×	×	×	×	×	×
	$B \equiv A, C \equiv A$	$D \equiv A$	$E \equiv A, F \equiv A$	$G \equiv A$	$G \equiv A, H \equiv A$	$H \equiv A$	輸出不同

圖3:BCD detector Implication Chart

G 與其他所有狀態輸出不同直接排除，可以發現無任何兩個狀態等效。8 個狀態已為minimum state machine，得到最簡state graph、state table。

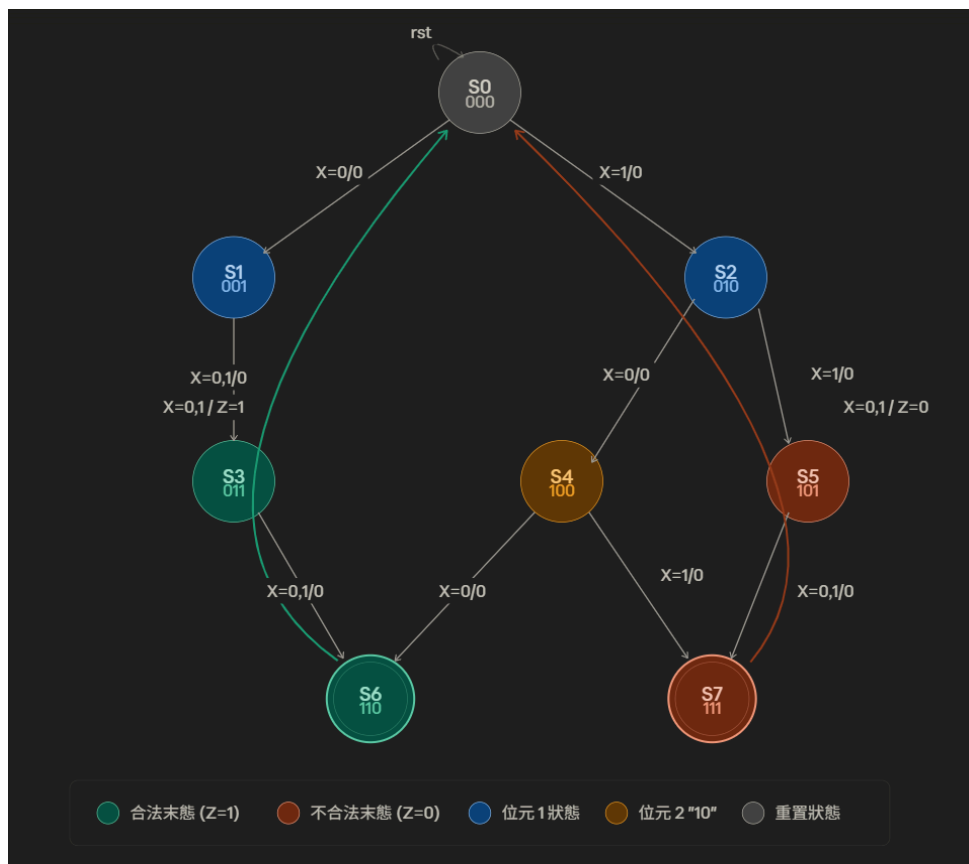


圖4:BCD detector state graph(最簡)

表3. BCD detector state table (minimum)

狀態	說明	NS (X=0)	NS (X=1)	Z (X=0)	Z (X=1)
A (S ₀)	Reset	B	C	0	0
B (S ₁)	收到 $b_3=0$	D	D	0	0
C (S ₂)	收到 $b_3=1$	E	F	0	0
D (S ₃)	收到 $b_3b_2 \in \{0x\}$	G	G	0	0
E (S ₄)	收到 $b_3b_2=10$	G	H	0	0
F (S ₅)	收到 $b_3b_2=11$	H	H	0	0
G (S ₆)	有效 BCD, Z=1	A	A	1	1
H (S ₇)	無效 BCD, Z=0	A	A	0	0

4. VHDL設計:

第一種方法:布林代數式

以 D type Flip-Flop進行設計, 8 個狀態需要 3 個 D 型正反器(Q2Q1Q0):

狀態	編碼 $Q_2Q_1Q_0$	狀態	編碼 $Q_2Q_1Q_0$
A	000	E	100
B	001	F	101
C	010	G	110
D	011	H	111

圖5:BCD detector State assignment

參考化簡過後的狀態表建立next state table:

表4. BCD detector next state table

狀態	Q_2	Q_1	Q_0	X	次態	D_2	D_1	D_0	Z
A	0	0	0	0	B=001	0	0	1	0
A	0	0	0	1	C=010	0	1	0	0
B	0	0	1	0	D=011	0	1	1	0
B	0	0	1	1	D=011	0	1	1	0
C	0	1	0	0	E=100	1	0	0	0
C	0	1	0	1	F=101	1	0	1	0
D	0	1	1	0	G=110	1	1	0	0
D	0	1	1	1	G=110	1	1	0	0
E	1	0	0	0	G=110	1	1	0	0
E	1	0	0	1	H=111	1	1	1	0
F	1	0	1	0	H=111	1	1	1	0
F	1	0	1	1	H=111	1	1	1	0
G	1	1	0	0	A=000	0	0	0	1
G	1	1	0	1	A=000	0	0	0	1
H	1	1	1	0	A=000	0	0	0	0
H	1	1	1	1	A=000	0	0	0	0

可以發現只有狀態為G時會輸出1(未化簡狀態表綠色圈選範圍), 參考next state table, 繪製卡諾圖並化簡得出 $D_2 \sim D_0$ 、 Z 的布林代數式:

$Q_2 \backslash Q_1 \backslash Q_0 X$	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	0	0	0	0
10	1	1	1	1

$$D_2 = \overline{Q_2}Q_1 + Q_2\overline{Q_1}$$

$Q_2 \backslash Q_1 \backslash Q_0 X$	00	01	11	10
00	0	1	1	1
01	0	0	1	1
11	0	0	0	0
10	1	1	1	1

$$D_1 = Q_2\overline{Q_1} + \overline{Q_2}Q_0 + \overline{Q_2}\overline{Q_1}X$$

$Q_2 \backslash Q_1 \backslash Q_0 X$	00	01	11	10
00	1	0	1	1
01	0	1	0	0
11	0	0	0	0
10	0	1	1	1

$$D_0 = \overline{Q_1}Q_0 + \overline{Q_2}Q_1\overline{X} + \overline{Q_2}Q_1\overline{Q_0}X + Q_2\overline{Q_1}X$$

$Q_2 \backslash Q_1 \backslash Q_0 X$	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	0	0
10	0	0	0	0

$$Z = Q_2 \cdot Q_1 \cdot \overline{Q_0}$$

圖6:BCD detector 卡諾圖化簡

得出表示式:

$$D_2 = Q_2'Q_1 + Q_2Q_1' \quad (\text{即 } Q_2 \oplus Q_1, \text{ 不依賴 } Q_0 \text{ 或 } X)$$

$$D_1 = Q_2Q_1' + Q_2'Q_0 + Q_2'Q_1'X$$

$$D_0 = Q_1'Q_0 + Q_2'Q_1'X' + Q_2'Q_1Q_0'X + Q_2Q_1'X$$

$$Z = Q_2 \cdot Q_1 \cdot Q_0' \quad (\text{僅狀態 G 時 } Z=1)$$

VHDL與分析:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity bcd_detector is
    Port (
        clk      : in  STD_LOGIC;           -- 系統時脈 (Clock)
        X        : in  STD_LOGIC;           -- 1-bit 序列輸入，每次接收 1 個 bit (MSB 先進)
        Z        : out STD_LOGIC;           -- 1-bit 輸出：當偵測到合法 BCD 碼時，Z 輸出 1
        state_out : out STD_LOGIC_VECTOR(2 downto 0) -- 3-bit 輸出：用來在模擬波形圖上觀察目前的內部狀態 (A, B, C)
    );
end bcd_detector;
architecture boolean_arch of bcd_detector is

    -- 【現態暫存器】
    -- 宣告 3 個 D 型正反器 (Flip-Flop) 的輸出端 A, B, C
    -- 給定初始值 ':= '0''，當 FPGA 上電 (Power-on) 或模擬開始時，狀態會自動初始化為 S0 (000)。
    -- 省去 Reset 接腳。
    signal A, B, C : STD_LOGIC := '0';

    -- 【次態訊號】
    -- 宣告 3 個 D 型正反器的輸入端 DA, DB, DC
    -- 由組合邏輯電路計算出來，準備在下一個 Clock 觸發時寫入 A, B, C
    signal DA, DB, DC : STD_LOGIC;

begin

    -- =====
    -- 第一部分：【次態邏輯】 (Combinational Logic)
    -- 根據「目前的狀態 (A, B, C)」與「當前的輸入 (X)」來決定下一個狀態
    -- 直接將卡諾圖化簡後的布林代數式轉換為 VHDL 邏輯閘語法
    -- =====

    -- DA
    -- 對應公式:  $DA = A'B + AB'$  (A XOR B)
    DA <= (not A and B) or (A and not B);

    -- DB
    -- 對應公式:  $DB = AB' + A'C + A'B'X$ 
    DB <= (A and not B)
        or (not A and C)
        or (not A and not B and X);

    -- DC
    -- 對應公式:  $DC = B'C + A'B'X' + AB'X + A'BC'X$ 
    DC <= (not B and C)
        or (not A and not B and not X)
        or (A and not B and X)
        or (not A and B and not C and X);

    -- =====
    -- 第二部分：【輸出邏輯】 (Combinational Logic)
    -- =====

    -- Mealy Machine 的特徵是：輸出會受到「現態」與「輸入」的影響。
    -- 根據你的設計，只有當狀態進入 G(S6) (A=1, B=1, C=0) 時，Z 才會輸出 1。
    Z <= A and B and not C;

    -- 將內部的現態訊號 (A, B, C) 使用 '&' (串接運算子) 打包成 3-bit 向量，
    -- state_out 方便 Debug 與觀察狀態機的跳轉過程。
    state_out <= A & B & C;

    -- =====
    -- 第三部分：【狀態暫存器】 (Sequential Logic)
    -- 在 Clock 邊緣更新狀態
    -- =====

    process(clk)
    begin
        -- 當偵測到 clk 的正緣 (Rising Edge) 時，執行以下動作：
        if rising_edge(clk) then
            -- 將組合電路算好的「次態 (DA, DB, DC)」正式寫入「現態 (A, B, C)」
            -- 完成一次狀態轉換 (State Transition)
            A <= DA;
            B <= DB;
            C <= DC;
        end if;
    end process;
end boolean_arch;

```

圖7:BCD detector VHDL(Boolean algebra)

為省去 Reset 版本 BCD 有效碼偵測器 (Mealy)，其架構主要由三個區塊協作：

第一部分是次態邏輯 (組合電路)，依據現態與輸入 X 的變化，透過化簡後的布林代數式 (DA、DB、DC) 即時算出下一個狀態；

第二部分是輸出邏輯，除了在特定現態 S6 (110) 時會讓 Mealy 輸出 $Z = 1$ 外，更利用串接運算子 (&) 將內部訊號實時導出至 state_out 向量，方便觀測與除錯；

第三部分則是狀態暫存器(循序電路)，負責在 clk rising edge到來時，將算好的次態存入現態暫存器(A、B、C)中。

另外，利用了 VHDL 的訊號初始值設定(:= '0')，讓電路在上電瞬間自動初始化為 S0 狀態，省去Reset 腳位。

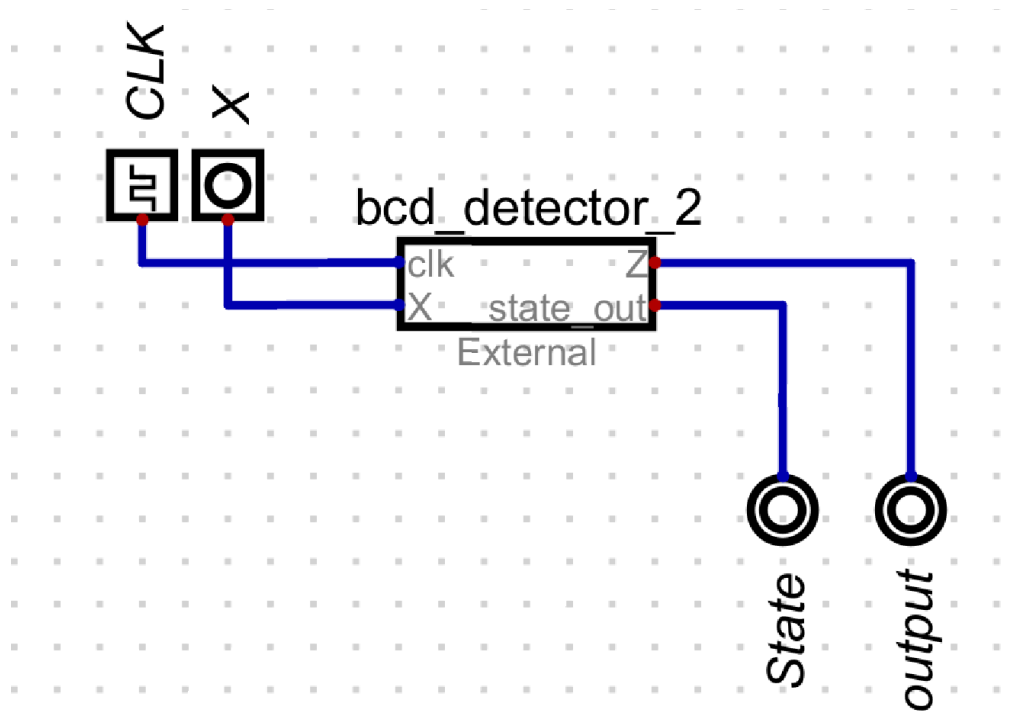


圖8:BCD detector VHDL模擬電路 (Boolean algebra)

第二種方法:**Process**

VHDL與分析:

```

--
-- 設計方式：使用內部暫存器與 4 狀態（S0~S3）進行序列資料比對
--
-- 功能說明：
-- 每接收 4 個位元後，判斷該 4 位元是否為合法 BCD 碼（0000~1001）
-- 輸入為 1-bit 序列輸入（預設 MSB 先進）
-- 在第 4 個狀態（S3）且結合當前輸入 X 時，若合法則輸出 Z = 1
--
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL; -- 必須引入此函式庫，才能對 STD LOGIC VECTOR 進行 "<=" 的數值大小比較

entity BCD_Detector is
    Port (
        clk      : in  STD_LOGIC;          -- 系統時脈輸入 (Clock)
        X        : in  STD_LOGIC;          -- 1-bit 序列資料輸入（每次 Clock 正緣讀取 1 bit）
        Z        : out STD_LOGIC;          -- 1-bit 判定輸出：當 4-bit 組合為合法 BCD 碼時輸出 1，其餘時間為 0
        state_bits : out STD_LOGIC_VECTOR(1 downto 0) -- 2-bit 狀態編碼輸出，用來給外部（或燈號）觀察目前在哪個狀態
    );
end BCD_Detector;

architecture Behavioral of BCD_Detector is
    -- 定義狀態機的 4 個狀態：分別代表接收到第 1、2、3、4 個位元的處理階段
    type state_type is (S0, S1, S2, S3);

    -- 宣告現態與次態訊號，並將現態初始值設為 S0（免去硬體 Reset 腳位）
    signal current_state : state_type := S0;
    signal next_state   : state_type;

    -- 【內部暫存器】用來儲存前 3 個讀入的序列位元，初始值為 "000"
    signal temp_reg      : STD_LOGIC_VECTOR(2 downto 0) := "000";

    -- 【組合向量】用來把前 3 個 bit 與當前第 4 個 bit (X) 拼接成完整的 4-bit 訊號線
    signal full_vector   : STD_LOGIC_VECTOR(3 downto 0);

begin
    --
    -- 循序邏輯製程 (Sequential Process)：負責狀態更新與資料暫存
    --
    process(clk)
    begin
        -- 當偵測到時脈正緣 (Rising Edge) 時觸發
        if rising_edge(clk) then
            -- 將次態 (next state) 正式鎖存入現態 (current state)
            current_state <= next_state;

            -- 【資料收集機制】依據「目前狀態」將輸入的 1-bit X 依序塞入暫存器的對應位置
            case current_state is
                when S0 =>
                    temp_reg(2) <= X; -- 在 S0 狀態讀入第 1 個 bit，當作最高位元 (MSB)
                when S1 =>
                    temp_reg(1) <= X; -- 在 S1 狀態讀入第 2 個 bit
                when S2 =>
                    temp_reg(0) <= X; -- 在 S2 狀態讀入第 3 個 bit
                when others =>
                    null; -- 在 S3 狀態時，第 4 個 bit 直接由當前的輸入 X 即時參與判斷，不需存入暫存器
            end case;
        end if;
    end process;

    --
    -- 資料拼接邏輯 (Concurrent Assignment)：
    -- 使用 '&' 運算子，將暫存的前 3 位元與當下輸入的第 4 位元 X 拼成 4-bit 訊號
    --
    full_vector <= temp_reg(2) & temp_reg(1) & temp_reg(0) & X;
end Behavioral;

```

```

-- 組合邏輯製程 (Combinational Process)：決定次態、輸出 Z 與狀態燈號
--
process(current state, X, full vector)
begin
    -- 【安全防護】給予所有輸出訊號預設值，防止 VHDL 合成出不必要的 Latch (鎖存器)
    Z <= '0';
    next state <= current state;
    state bits <= "00";

    -- 依據現態進行邏輯判斷
    case current state is
        -- 初始狀態：準備收第 1 個 bit
        when S0 =>
            next state <= S1;    -- 下個時脈跳往 S1
            state bits <= "00";  -- 輸出狀態編碼 00

        -- 狀態 1：準備收第 2 個 bit
        when S1 =>
            next state <= S2;    -- 下個時脈跳往 S2
            state bits <= "01";  -- 輸出狀態編碼 01

        -- 狀態 2：準備收第 3 個 bit
        when S2 =>
            next state <= S3;    -- 下個時脈跳往 S3
            state bits <= "10";  -- 輸出狀態編碼 10

        -- 狀態 3：準備收第 4 個 bit 的關鍵時刻 (判定點)
        when S3 =>
            next state <= S0;    -- 收集完 4 bits 後，下個時脈自動回到 S0 重新循環
            state bits <= "11";  -- 輸出狀態編碼 11

            -- 【BCD 核心判定】此時 full vector 已集滿 4 位元
            -- 因為 Mealy 輸出包含當前輸入 X (已透過 full vector 隱含)，所以在 S3 當下直接做數值比對
            if full vector <= "1001" then
                Z <= '1';      -- 數值在 0000 ~ 1001 (0~9) 之間：為合法 BCD 碼，輸出 1
            else
                Z <= '0';      -- 數值在 1010 ~ 1111 (10~15) 之間：為非法碼，輸出 0
            end if;

        -- 異常狀態處理 (防止電路卡死，提升電路魯棒性)
        when others =>
            next state <= S0;
            state bits <= "00";
            Z <= '0';
    end case;
end process;
end Behavioral;

```

圖9:BCD detector VHDL (Process)

本實驗比較了兩種不同設計思維的 BCD 偵測器架構。

前案布林代數式屬於「純狀態機」設計，因缺乏獨立儲存空間，必須透過大量狀態的分支 (共 8 個狀態) 來記憶歷史輸入位元，雖具備硬體資源 (正反器) 最精簡與邏輯閘延遲低的優點，但存在代數式可讀性低且硬體更動極其困難的缺陷；

而後案行為描述則採用現代數位系統標準的「控制器與資料路徑 (Control + Datapath)」架構，將狀態機化簡為僅需 4 個狀態 (S0~S3) 來推進時脈步進，並引入 3-bit 內部暫存器 (temp_reg) 專責串行資料的收集與拼接。此種設計雖然因配置了獨立暫存器而略微增加了正反器的硬體消耗量，卻大幅換取了直覺的程式可讀性 (直接使用數值比較運算子)，並賦予電路極佳的維護彈性，符合 FPGA 開發重視軟體維護效率趨勢。

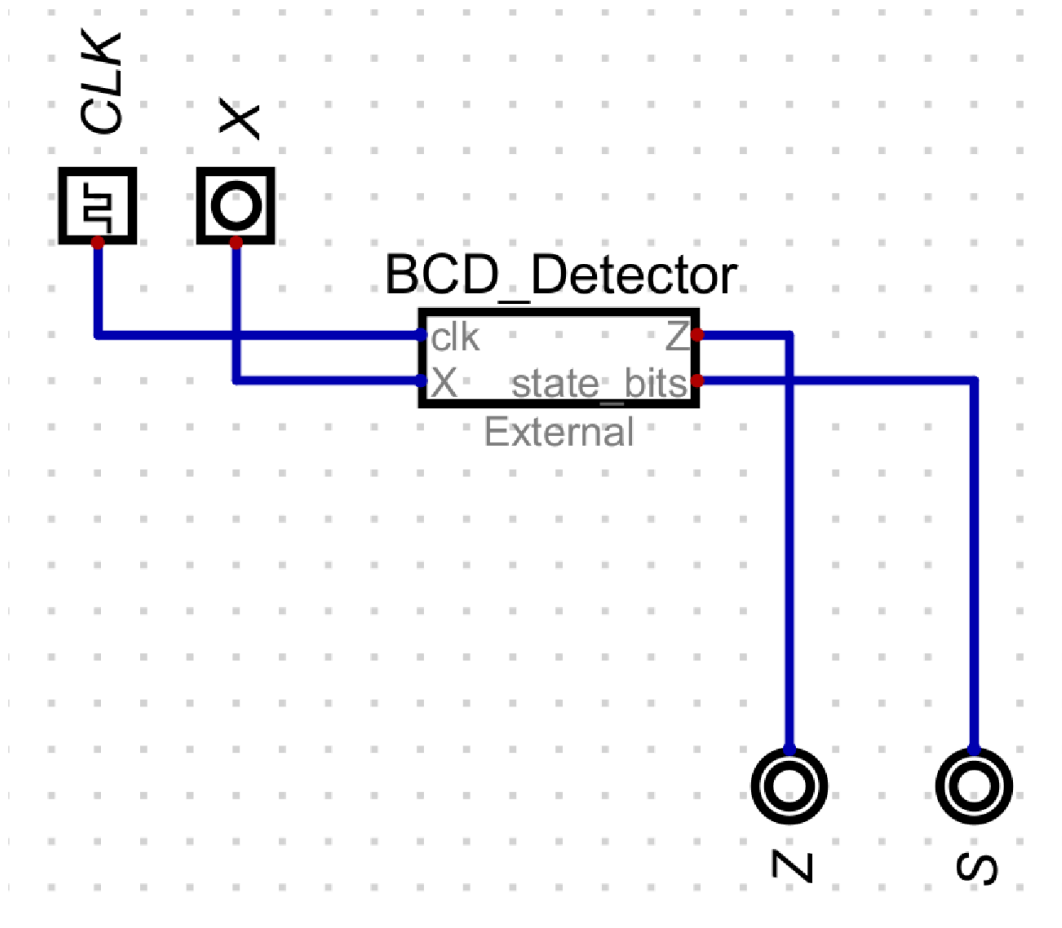


圖10:BCD detector VHDL模擬電路 (Process)

B. 作業2

本次設計程序採用 VHDL 實作一組串列加法器。在電路架構上，主要由全加器、移位暫存器與 D 型正反器構成。運作過程中，運算元的輸入資料透過移位暫存器，在每個時脈週期同步將最後一位元送入全加器；全加器即時計算出該位元的總和並輸出至移位暫存器最高位元。同時，全加器所產生的進位輸出會存至 D 型正反器中，並於下一個時脈的正緣作為下一位元加法的進位輸入。

第一步(狀態圖):根據(圖11)改畫出控制串列加法器的moore狀態機狀態圖，在狀態0時，等待開始訊號，一收到訊號，無論開始訊號有無切換回低電位，都必須將狀態全走過一次，等走到狀態5，強制將電路停止，直到開始訊號回到低電位才回到狀態0。

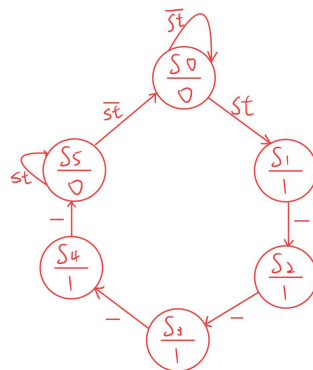


圖11:作業二moore狀態機狀態圖

第二步 Process方法:本次串列加法器的「控制電路」採用 VHDL 實作了一組 Moore 狀態機。將電路分為同步狀態更新與次態/輸出邏輯兩段 Process 撰寫。當接收到啟動訊號 (st = '1') 後, 狀態機會依序推進 4 個狀態("001" 到 "100")。這段期間內會持續輸出高電位 (sh = '1'), 驅動外部電路完成 4 bits 的逐位元加法。等待與重置機制:完成 4 次移位後, 系統會進入等待狀態("101")並關閉輸出。必須等待外部的啟動訊號 st 降回 '0', 系統才會重置回初始狀態, 確保每次運算指令不會互相干擾。程式碼中定義了訊號的初始預設值, 並加入了 when others 的防呆機制。這能避免電路產生非預期的鎖存器 (Latch), 並確保系統在異常時能自動回到安全狀態, 防止當機死鎖。

```
1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity StateMachine is
5.     Port (
6.         clk : in  std_logic; -- 時脈輸入
7.         st  : in  std_logic; -- 啟動訊號
8.         sh  : out std_logic -- 狀態機輸出(Right Shift)
9.     );
10. end StateMachine;
11.
12. architecture Behavioral of StateMachine is
13.     -- 設定初始值為 "000"
14.     signal current_state : std_logic_vector(2 downto 0) := "000";
15.     signal next_state   : std_logic_vector(2 downto 0);
16. begin
17.
18.     -- 第一段 Process: 同步時序邏輯 (Sequential Logic)
19.     -- 每個 Clock 正緣時, 更新當前狀態
20.     process(clk)
21.     begin
22.         if rising_edge(clk) then
23.             current_state <= next_state; -- 將下一個狀態更新為當前狀態
24.         end if;
25.     end process;
26.
27.     process(current_state, st)
28.     begin
29.         next_state <= current_state;
30.         sh <= '0';
31.
32.         case current_state is
33.             -- 狀態 0: 等待啟動訊號
34.             when "000" =>
35.                 sh <= '0'; -- 閒置時輸出為 0
36.                 if st = '0' then
37.                     next_state <= "000"; -- 若沒有啟動訊號, 保持在狀態 0
38.                 else
39.                     next_state <= "001"; -- 若 st = '1', 進入狀態 1 開始動作
40.                 end if;
41.
42.             -- 狀態 1: 輸出拉高, 準備進入下一狀態
```

```

43. when "001" =>
44.     sh <= '1';          -- 輸出變為 1 (第 1 個週期)
45.     -- 無論 st 為何, 下一個 Clock 都無條件進入狀態 2
46.     if st = '0' then
47.         next_state <= "010";
48.     else
49.         next_state <= "010";
50.     end if;
51.
52. -- 狀態 2: 維持輸出
53. when "010" =>
54.     sh <= '1';          -- 輸出維持 1 (第 2 個週期)
55.     -- 無條件進入狀態 3
56.     if st = '0' then
57.         next_state <= "011";
58.     else
59.         next_state <= "011";
60.     end if;
61.
62. -- 狀態 3: 維持輸出
63. when "011" =>
64.     sh <= '1';          -- 輸出維持 1 (第 3 個週期)
65.     -- 無條件進入狀態 4
66.     if st = '0' then
67.         next_state <= "100";
68.     else
69.         next_state <= "100";
70.     end if;
71.
72. -- 狀態 4: 維持輸出
73. when "100" =>
74.     sh <= '1';          -- 輸出維持 1 (第 4 個週期)
75.     -- 無條件進入狀態 5
76.     if st = '0' then
77.         next_state <= "101";
78.     else
79.         next_state <= "101";
80.     end if;
81.
82. -- 狀態 5 (end): 輸出結束, 等待輸入訊號歸零
83. when "101" =>
84.     sh <= '0';          -- 輸出恢復為 0
85.     if st = '0' then
86.         next_state <= "000"; -- 若 st 已經降回 0, 則回到初始狀態 0
87.     else
88.         next_state <= "101"; -- 若 st 仍然為 1, 則停留在狀態 5 等待
89.     end if;
90.
91. -- 其他未定義狀態

```

```

92.         when others =>
93.             next_state <= "000"; -- 強制回到狀態 0
94.             sh <= '0';          -- 確保輸出為安全值 0
95.         end case;
96.     end process;
97.
98. end Behavioral;

```

第三步 電路組合：將第二步所做成的控制電路與之前所做的移位暫存器、一位元加法器組合(圖12)，唯一不一樣在於第一個移位暫存器多了一個QZ用來輸出暫存器裡的值

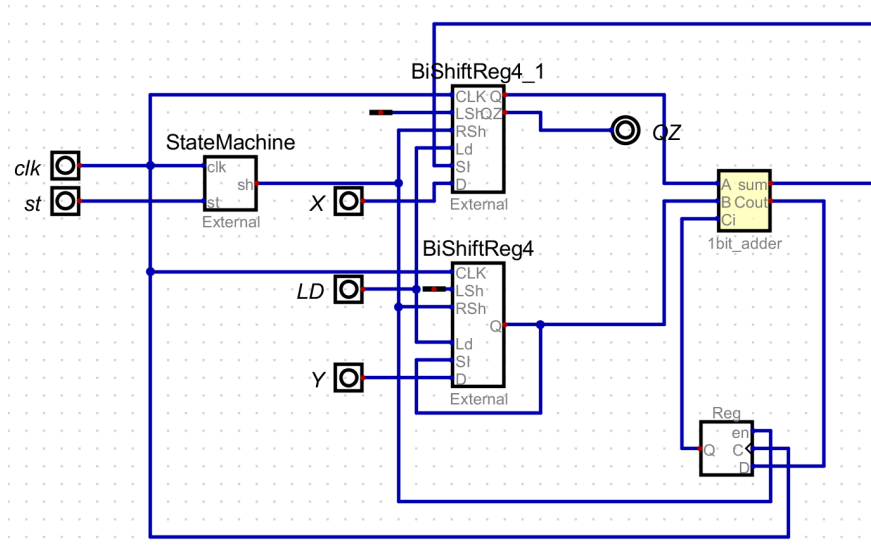


圖12:第二題電路

III. 模擬驗證

A. 作業1

兩個設計在State表示意義不同，布林代數方法的State(顯示 3-bit)其輸入情形的編號；行為級描述(Process)(顯示 2-bit)則表示計數器，代表已輸入幾個位元。

第一種方法:布林代數式(參考圖4.)

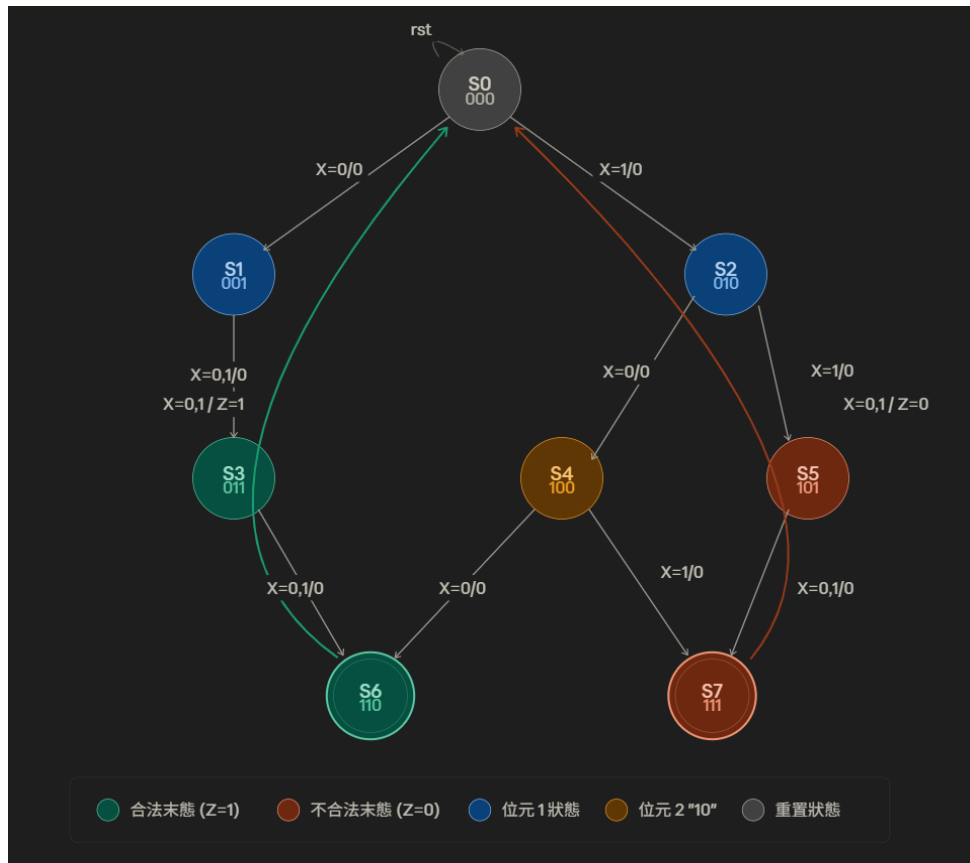


圖4:BCD detector state graph(最簡)

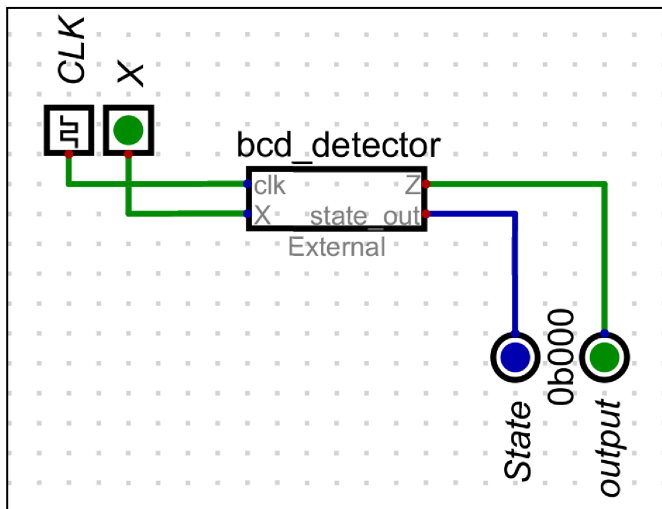


圖13:Input:0 / Output:0 / State:000(S0) (初始化) (0)

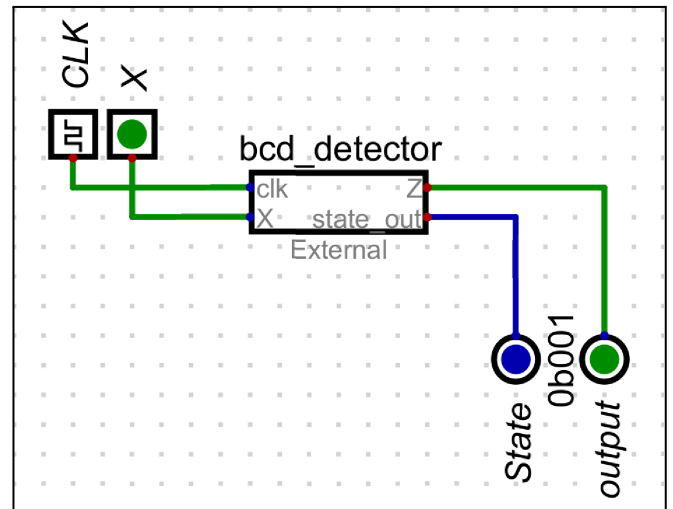


圖14:Input:0 / Output:0 / State:001(S1) (00)

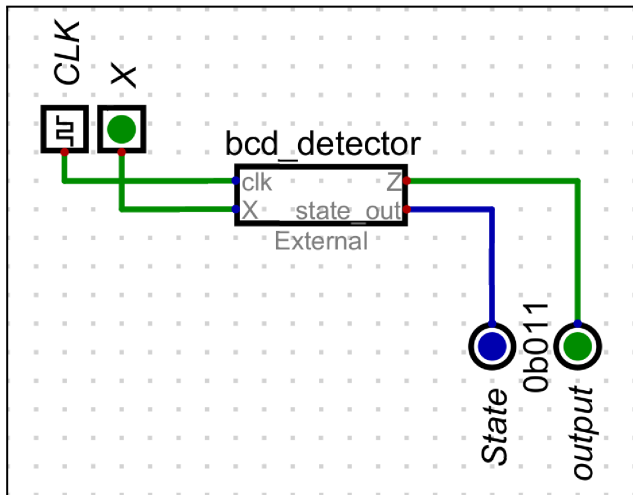


圖15: Input: 0 / Output: 0 / State: 011(S3) (000)

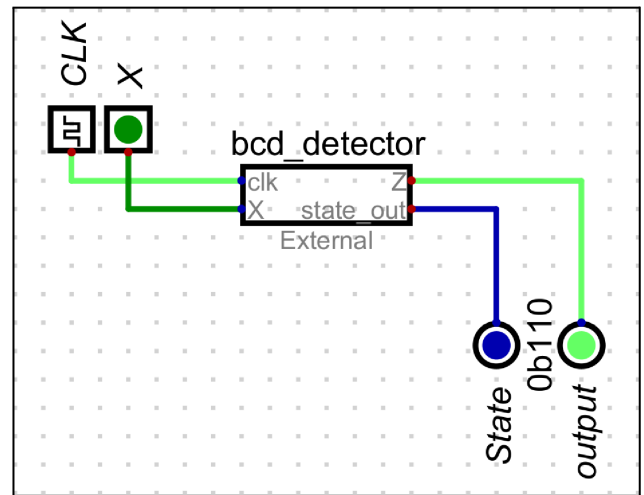


圖16: Input: 1 / Output: 1 / State: 110(S6) (0001)(V)

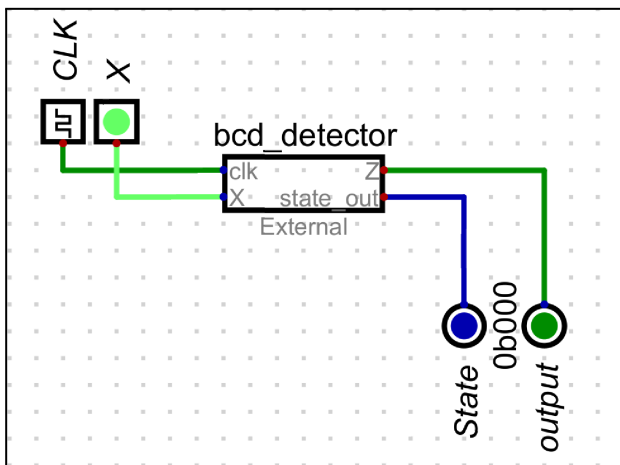


圖17: Input: 1 / Output: 0 / State: 000(S0) (1)

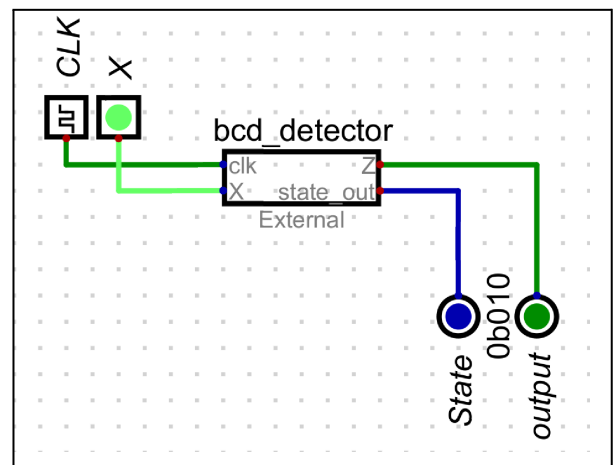


圖18: Input: 1 / Output: 0 / State: 010(S2) (11)

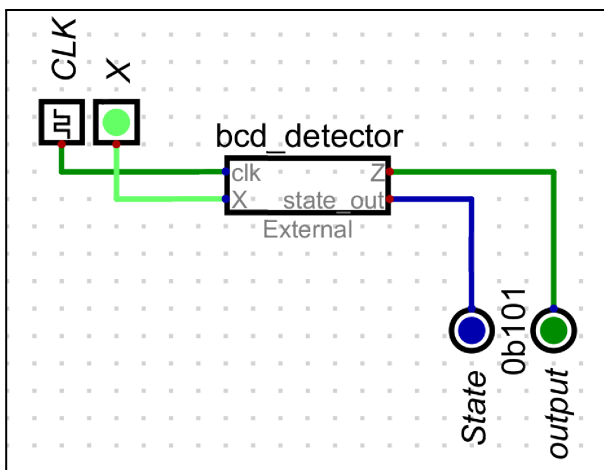


圖19: Input: 1 / Output: 0 / State: 101(S5) (111)

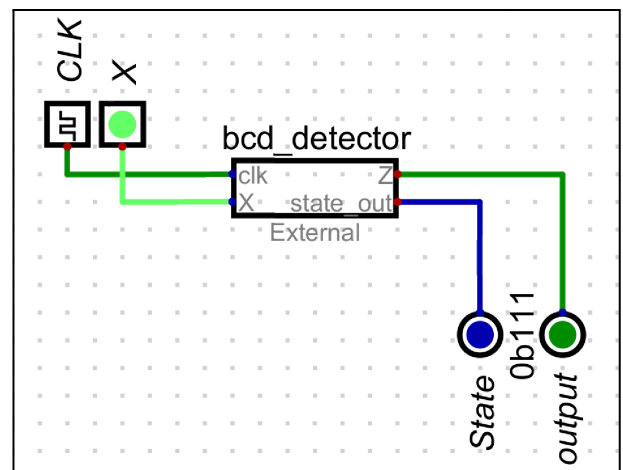


圖20: Input: 1 / Output: 0 / State: 111(S7) (1111)(X)

第二種方法: **Process**

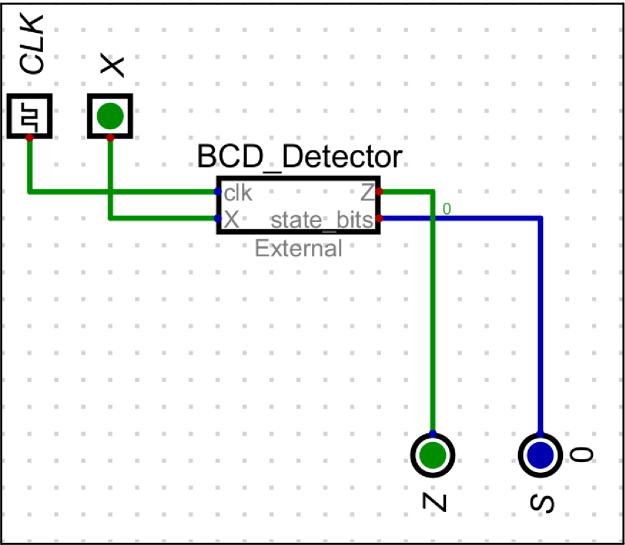


圖21: Input:0 / Output:0 / State:0(S0) (初始化) (輸入0次)
(0)

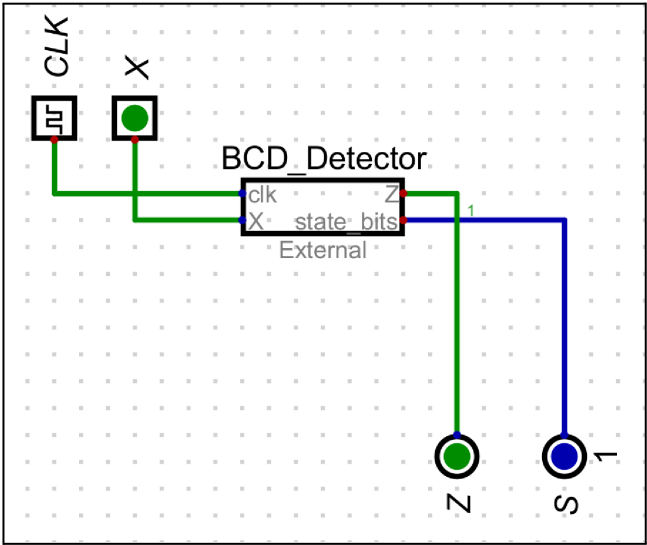


圖22: Input:0 / Output:0 / State:1(S1) (輸入1次)
(00)

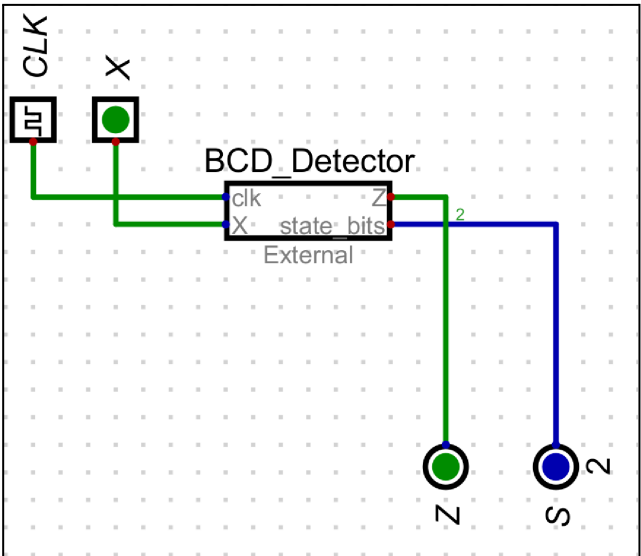


圖23: Input:0 / Output:0 / State:2(S2) (輸入2次)
(000)

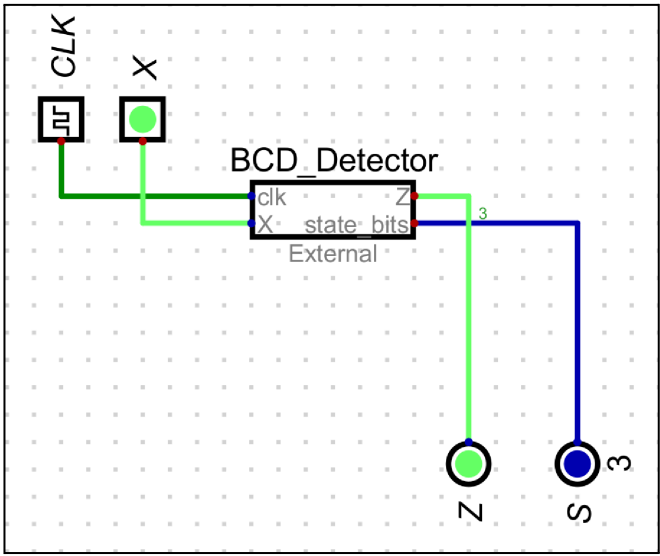


圖24: Input:1 / Output:1 / State:3(S3) (輸入3次)
(0001) (1還沒過CLK)

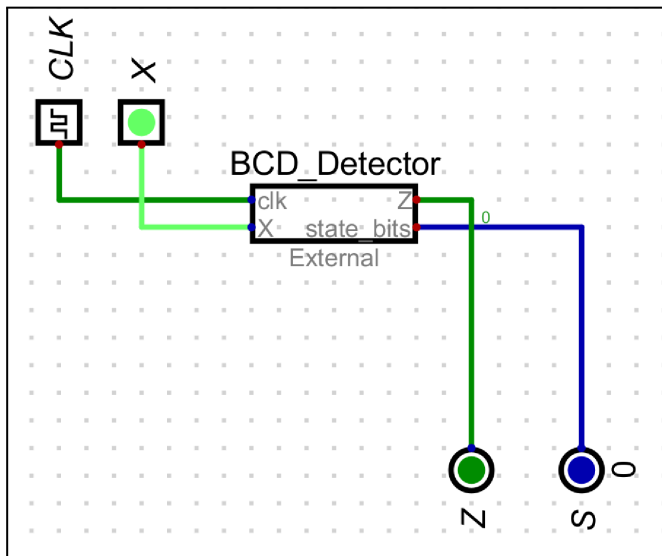


圖25: Input: 1 / Output: 0 / State: 0(S0) (初始化) (輸入0次)
(1)

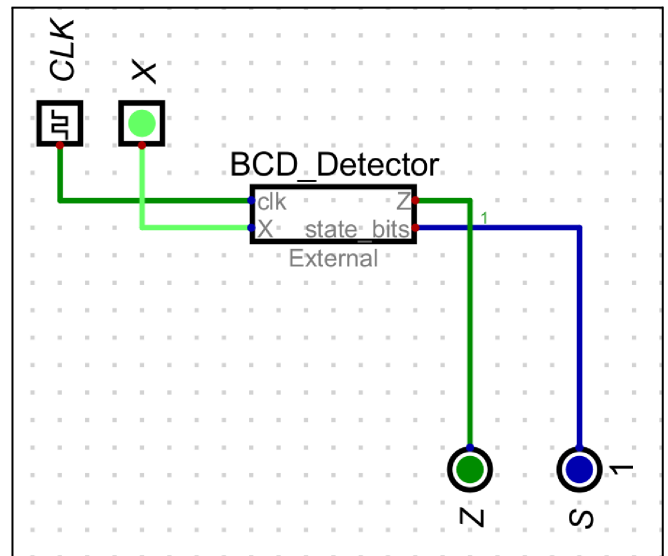


圖26: Input: 1 / Output: 0 / State: 1(S1) (輸入1次)
(11)

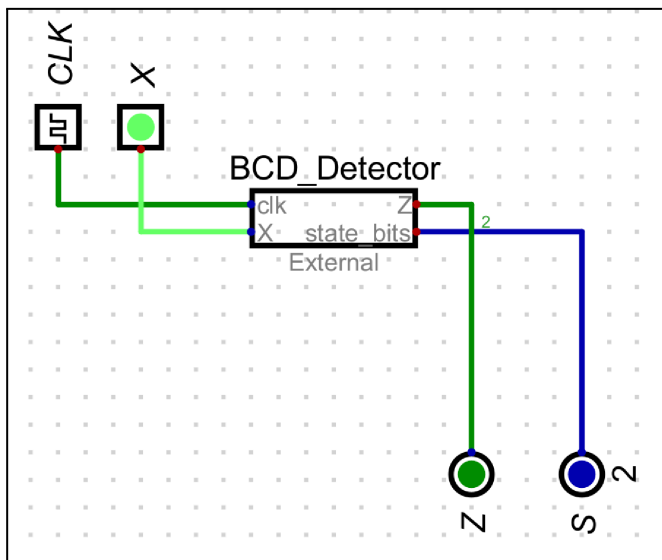


圖27: Input: 1 / Output: 0 / State: 2(S2) (輸入2次)
(111)

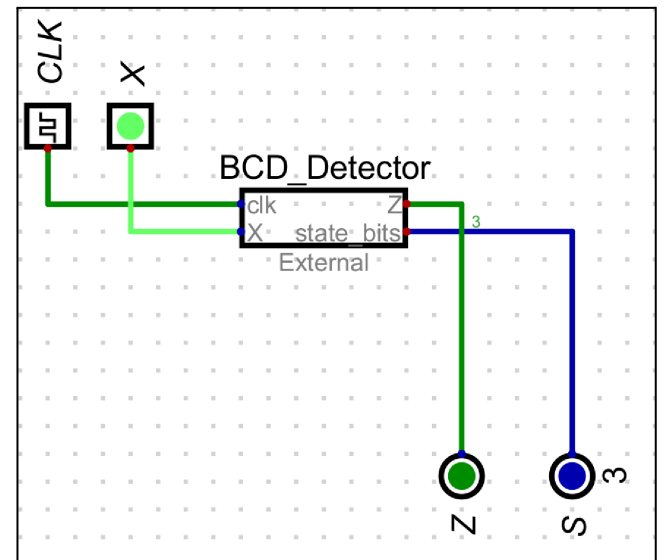


圖28: Input: 1 / Output: 0 / State: 3(S3) (輸入3次)
(1111) (1還沒過CLK)

B. 作業2

為驗證本組設計之四位元串列加法器與 Moore 型控制器的正確性，我們於軟體中給予時脈訊號 (clk) 與啟動訊號 (St)，並設定兩組 4 位元數值作為加數與被加數進行模擬測試。圖29至圖34模擬如下：

1. Moore 狀態機控制邏輯驗證：

- 閒置與啟動：如圖(圖29)所示，系統初始停留在閒置狀態S。當給予啟動訊號 $St = 1$ 且時脈上升緣到來時，狀態機轉至運算狀態S1。

- **Moore** 輸出特性:在狀態S1至S4期間, 控制移位暫存器的移位致能訊號皆被正確拉高為 1。
- 運算終止:經過 4 個時脈週期後, 狀態機會進入停止狀態S5, 同時 Sh 訊號降為 0, 鎖住最終的運算結果, 完成自動控制循環。

2. 逐位元運算與移位資料流驗證：

- 進位儲存: 每一個時脈週期, 全加器算出的進位輸出皆能暫存至D型暫存器, 並作為下一個時脈週期的進位輸入。
- 移位與總和擷取: 在 $Sh = 1$ 的四個時脈期間, 加數與被加數暫存器如預期地向右移位。全加器逐位元算出的總和也依序從最高位元推入結果暫存器中。

3. 運算結果正確性：

$A = [0111]$, $B = [0011]$ 。經過 4 次完整的移位與相加週期後, 觀察最終的輸出暫存器, 其數值正確顯示為 $[1010]$, 且狀態正確。

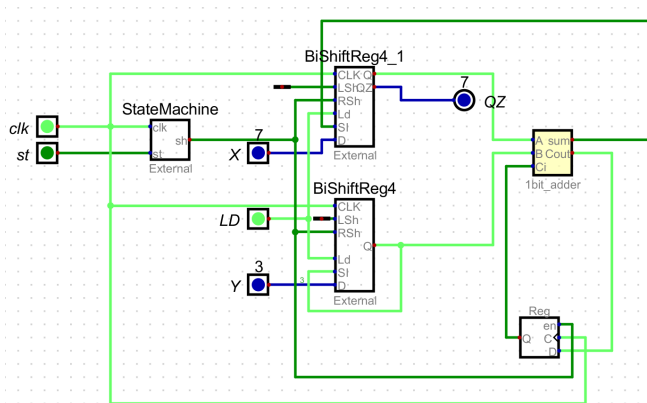


圖29:S0 將初始值load進移位暫存器

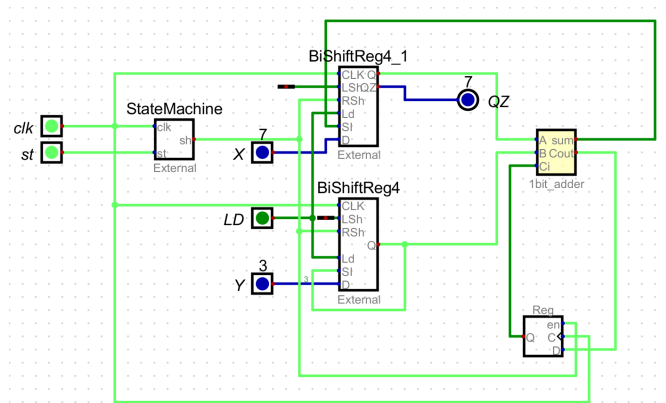


圖30:S1 將最後一位元相加為0

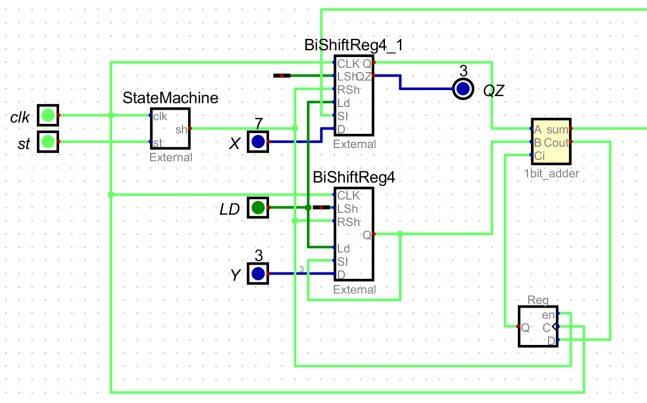


圖31:S2 將倒數第二位元相加為1, 並將前一步0輪回移位暫存器第一位, 輸出從0111變為0011

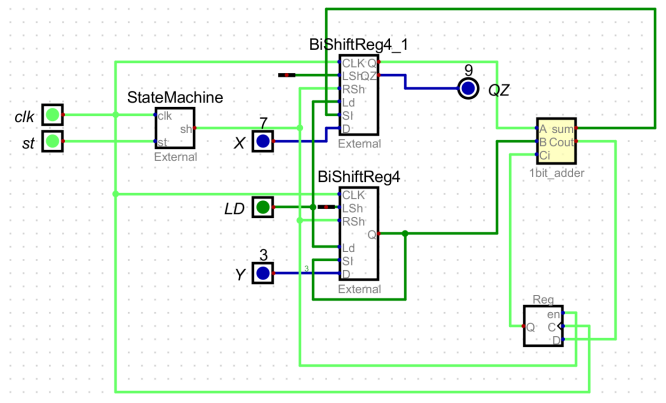


圖32:S3 將倒數第三位元相加為0, 並將前一步1輪回移位暫存器第一位, 輸出從0011變為1001

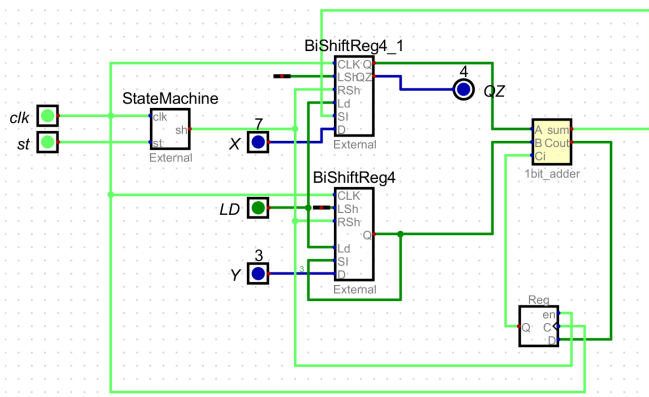


圖33:S4 將最高位元相加為1, 並將前一步0輪回移位暫存器第一位, 輸出從1001變為0100

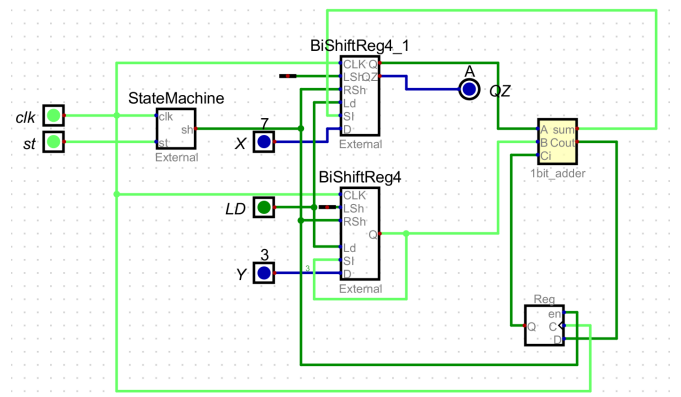


圖34:S5 將前一步1輪回移位暫存器第一位, 輸出從0100變為1010

IV. 實驗結果與成果討論

A. 作業1

以Process方式進行燒錄, 以KEY3作為CLK;SW0作為 Input X

並以 LEDR2~LEDR1作為State;LEDR0作為Output Z 進行實體測試。

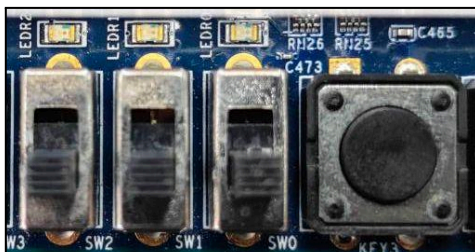


圖35:Input:0 / Output:0 / State:0(S0) (初始化) (輸入0次)
(0)

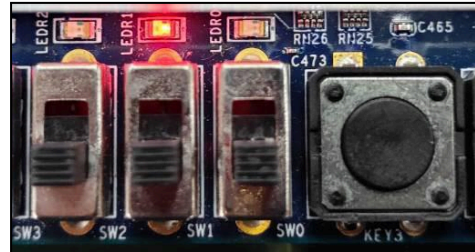


圖36:Input:0 / Output:0 / State:1(S1) (輸入1次)
(00)

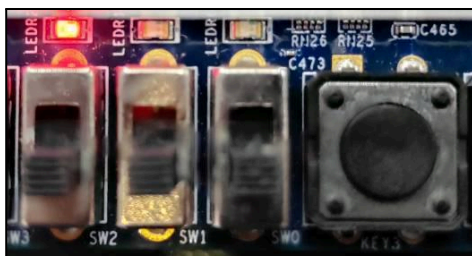


圖37:Input:0 / Output:0 / State:2(S2) (輸入2次)
(000)

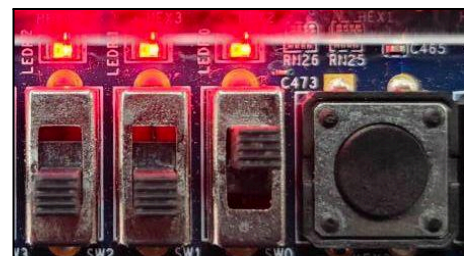


圖38:Input:1 / Output:1 / State:3(S3) (輸入3次)
(0001) (1還沒過CLK)

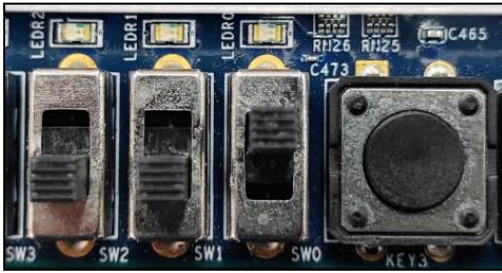


圖39:Input:1 / Output:0 / State:0(S0) (初始化) (輸入0次)
(1)

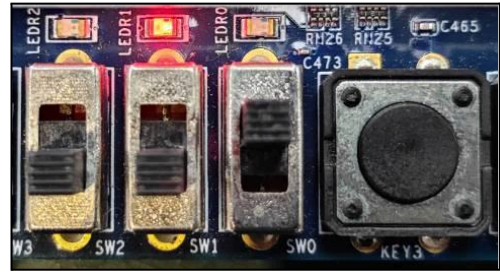


圖40:Input:1 / Output:0 / State:1(S1) (輸入1次)
(11)

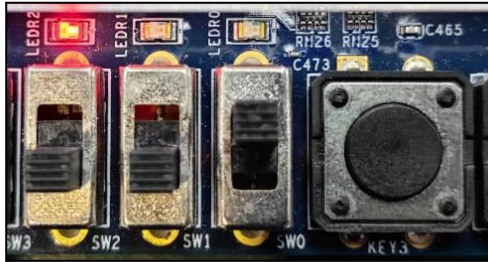


圖41:Input:1 / Output:0 / State:2(S2) (輸入2次)
(111)

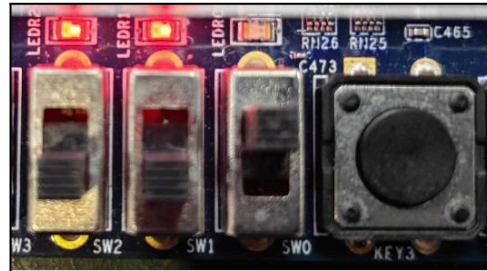


圖42:Input:1 / Output:0 / State:3(S3) (輸入3次)
(1111)(1還沒過CLK)

B. 作業2

燒錄完成後，以指撥開關SW9作為st開始訊號，SW8作為LD輸入初始值，SW7~SW4作為被加數輸入，SW3~SW0作為加數輸入，另外以按壓開關KEY3作為時脈輸入clk，並以LEDR3~LEDR0作為輸出Q。我們將模擬時的測資實際操作，觀察發現只有在 CLK 開關按下的瞬間，LED 燈號才會依照狀態圖改變；在 CLK 維持不變的期間，輸出皆保持不變。實體驗證結果與軟體模擬完全吻合。

1. 第一步:將初始值load進移位暫存器。
2. 第二步:將最後一位元相加值0輪回移位暫存器第一位，輸出從0111變為0011。
3. 第三步:將倒數第二位元相加值1輪回移位暫存器第一位，輸出從0011變為1001。
4. 第四步:將倒數第三位元相加值0輪回移位暫存器第一位，輸出從1001變為0100。
5. 第五步:將最高位元相加值1輪回移位暫存器第一位，輸出從0100變為1010。

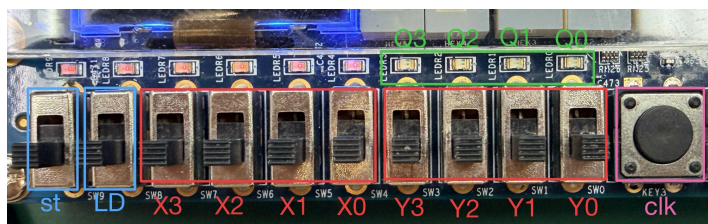


圖43:作業二腳位對應

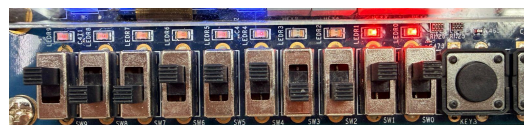


圖44:將初始值load進移位暫存器

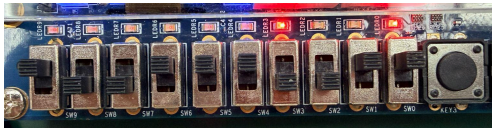


圖45:將第一步0輸回移位暫存器第一位, 輸出從0111變為0011

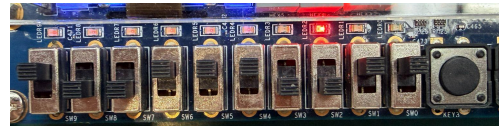


圖46:將前一步1輸回移位暫存器第一位, 輸出從0011變為1001

圖47:將前一步0輸回移位暫存器第一位, 輸出從1001變為0100

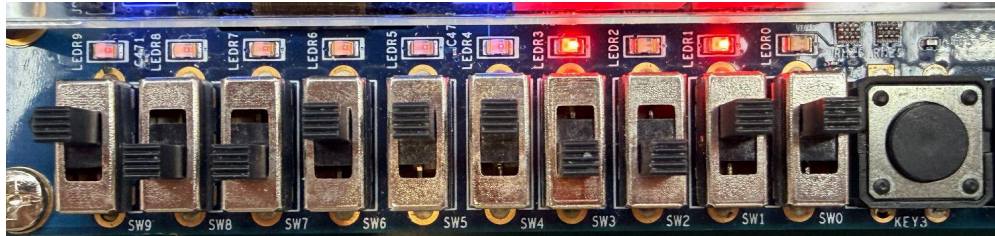


圖48:將前一步1輸回移位暫存器第一位, 輸出從0100變為1010

V. 實驗心得

組員1:這次的實驗是沒學過的, 雖然麻煩一點, 但還是有順利做出來。

組員 2:第一次使用梯形圖進行狀態省略, 搞懂後就沒那麼困難了, 另外也可以發現狀態簡化跟以行為模式去寫VHDL概念大不相同, 應該就是上課說的越複雜控制系統越不適合用此方法簡化?

參考文獻

- [1] DLE 11 數位邏輯實習 DLE 11 REDUCTION OF STATE TABLES AND ASSIGNMENTS, SERIAL ADDER